四川大学计算机学院、软件学院

实验报告

学号：2024141460445 姓名：许盛凯 专业：计算机科学与技术 班级：周二 34 节 第 8 周

课程名称	计算机网络课程设计	实验课时	第 3—4 节
实验项目	Web 服务器搭建与 HTTP 协议分析	实验时间	2026 年 10 月 13 日
实验目的	本实验为综合型实验，具体目的如下： 1. 配置 Apache，理解根目录、默认文档和监听端口。 2. 通过 HTTP 访问首页，掌握 URL 与文件的对应关系。 3. 分析实际 HTTP 请求、响应、状态码和各类首部。 4. 比较首次访问与普通刷新，理解条件 GET 和缓存验证。		
实验环境	系统：Arch Linux，内核 7.2.9-arch1-1，x86_64。 硬件：Intel Core i9-13900HX；内存 31,775 MiB。 服务器：Apache HTTP Server 2.4.69（Unix）。 浏览器：Chromium 153.0.8010.52，语言 zh-CN。 抓包：Wireshark/TShark 4.7.4，dumpcap，接口 any。 本地站点：`http://127.0.0.1:8086/`。 实验站点：`gaia.cs.umass.edu`，TCP 80 端口。		
实验内容（算法、程序、步骤和方法）	**一、任务 1：Apache 的获取、配置与验证** 1. **选择 Apache**。本机为 Arch Linux，选择 Apache 而非 IIS。原 `win32` 压缩包不适用于 Linux，改从 Arch 官方 extra 仓库获取 Apache 2.4.69、APR 和 APR-util。 2. **验证并隔离运行**。用 Arch 公钥环核验各包独立签名，均有效，再解包到独立目录；`LD_LIBRARY_PATH` 指向对应依赖，不启用系统服务。来源、哈希和签名记录见`evidence/software-provenance.txt`。 3. **创建默认首页**。制作 UTF-8 的 `index.html`，标题为“计算机网络课程设计——实验 1”，写明姓名、学号、班级及实验内容。`experiment/httpd.conf` 设置根目录与默认文档。 4. **检查与访问**。运行 `httpd -f <配置路径> -t`，输出 `Syntax OK`，再前台启动。`ss -ltn` 确认监听 `127.0.0.1:8086`，GET 根路径得到 200，浏览器显示自建首页。		

（接上）实验内容（算法、程序、步骤和方法）

二、服务器配置与自建首页的实际结果

```
Arch Linux | Apache HTTP Server | 实验1实际配置验证

$ /tmp/opencode/nc-cl-work/runtime/usr/bin/httpd -v
Server version: Apache/2.4.69 (Unix)
Server built:   Oct  1 2026 19:32:23

$ /tmp/opencode/nc-cl-work/runtime/usr/bin/httpd -f /home/assumeengage/NC_cl/实验1_Overleaf/experiment/httpd.conf -t
Syntax OK

$ 读取实际 httpd.conf 中的关键指令
Listen 127.0.0.1:8086
ServerName 127.0.0.1:8086
DocumentRoot "/home/assumeengage/NC_cl/实验1_Overleaf/experiment/www"
<Directory "/home/assumeengage/NC_cl/实验1_Overleaf/experiment/www">
    Require all granted
DirectoryIndex index.html
AddDefaultCharset UTF-8
KeepAlive On
KeepAliveTimeout 5

$ ss -ltn: 本实验监听端口
State  Recv-Q Send-Q             Local Address:Port  Peer Address:Port
LISTEN 0      511                    127.0.0.1:8086       0.0.0.0:*

$ 浏览器实际访问日志（末尾记录）
127.0.0.1 - - [13/Oct/2026:09:04:05 -0400] "GET / HTTP/1.1" 200 1819
127.0.0.1 - - [13/Oct/2026:09:04:06 -0400] "GET /favicon.ico HTTP/1.1" 404 236
127.0.0.1 - - [13/Oct/2026:09:04:32 -0400] "GET / HTTP/1.1" 200 1819

证据来自本机实际运行。按 Enter 关闭此窗口。
```

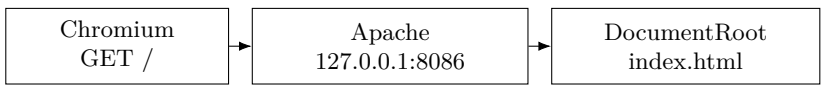
图 1　Apache 版本、关键配置、语法检查与监听状态

`Listen 127.0.0.1:8086` 仅在本机回环地址监听；`DocumentRoot` 确定网站根目录，`DirectoryIndex index.html` 选择默认首页。`Require all granted` 授权访问指定目录；`Options -Indexes` 禁止目录索引，`AllowOverride None` 禁止.htaccess 覆盖。



图 2　真实浏览器访问本次站点，地址栏及个人信息可见

实际根目录为原实验工程的实验 `1_Overleaf/experiment/www`；`/` 映射到该目录，再返回 `index.html`，不是双击文件或通过 `file://` 打开。HTTP 默认端口为 80，本次使用 8086，故 URL 必须含`:8086`；地址栏、Apache 响应标识和 200 访问日志相互印证。



（接上）实验内容（算法、程序、步骤和方法）

三、任务 2：基本 HTTP 抓包过程与证据

在隔离浏览器中清除实验资源缓存，先启动 dumpcap，再明确以 HTTP 访问指导书指定页面：

`http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file1.html`

捕获过滤器为 `host 128.119.245.12 and tcp port 80`；显示过滤器为 `http`。抓包共 38 帧，10 帧识别为 HTTP；帧 8 与帧 12 构成目标页的完整请求和响应，不把 `favicon.ico` 的交换当作目标页。

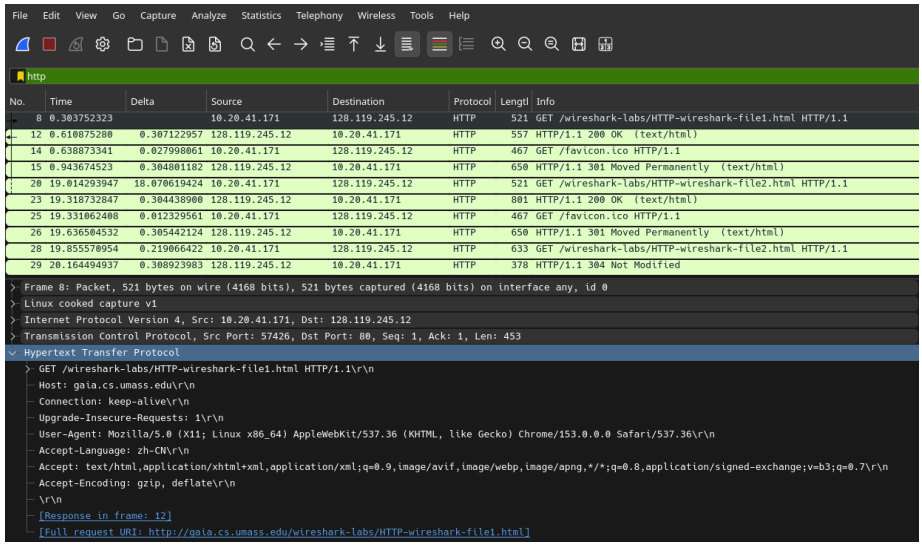


图 3　file1 的 GET 请求，帧 8

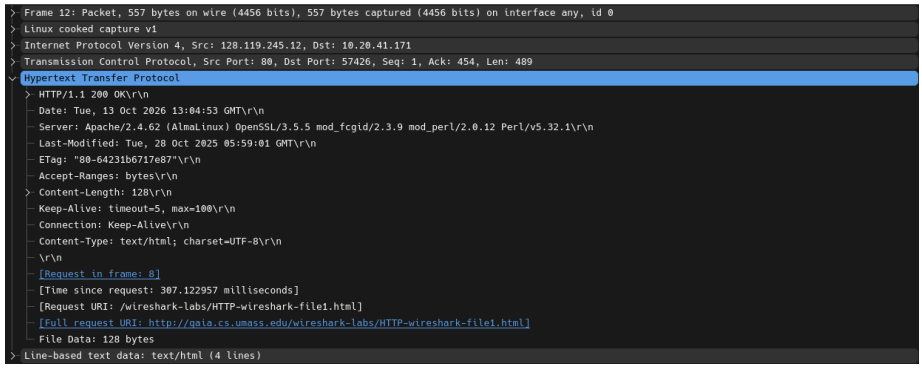


图 4　file1 的 200 OK 响应，帧 12

图 3 给出请求行、语言、编码与响应关联；图 4 给出版本、状态码、类型、长度和最后修改时间。帧 8 和 12 属于 `tcp.stream == 0`。完整截图与原始evidence/http-experiment.pcapng 均随工程保存。

图 3—8 均来自同一次真实抓包；裁剪图只去除无关窗格和空白，未改写字段，未以示例数据替代本机记录。

（接上）实验内容（算法、程序、步骤和方法）

四、任务 3：清除缓存后的首次访问

`http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html`

首次访问前通过浏览器的缓存清除接口清空缓存，再正常导航到上述原始 URL；未添加查询参数，也未手工注入验证器。首次请求为帧 20，响应为帧 23。

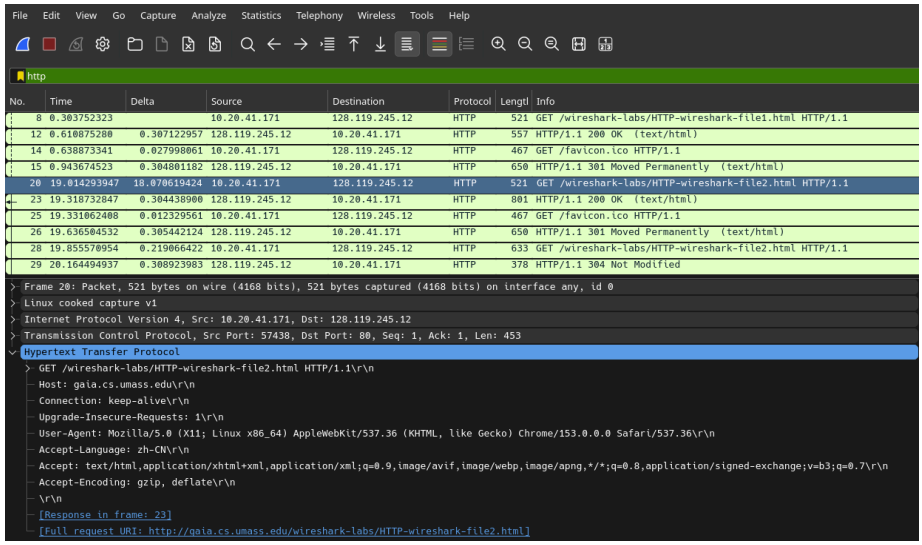


图 5　file2 首次 GET，帧 20，无缓存验证字段

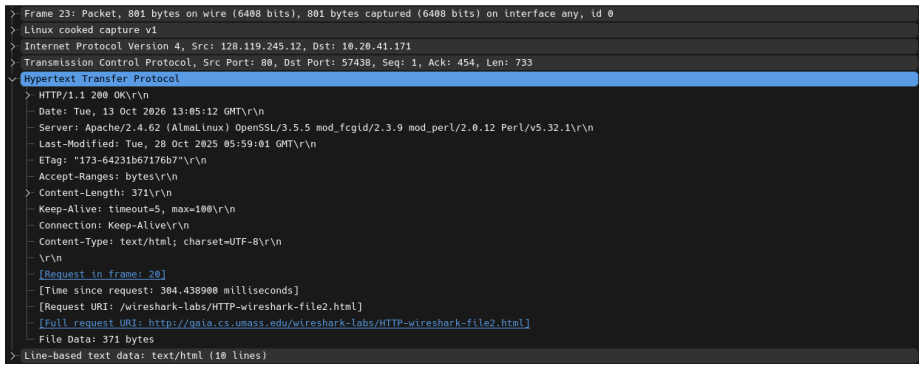


图 6　首次响应，帧 23，200 OK 及 371 字节实体

图 5 的完整请求没有 `If-Modified-Since` 或 `If-None-Match`。图 6 的响应给出 `Last-Modified` 和 `ETag`，供下一次缓存验证使用。

该连接的 TCP 四元组为客户端 `10.20.41.171:57438` 与服务器 `128.119.245.12:80`，属于 `tcp.stream == 1`。

（接上）实验内容（算法、程序、步骤和方法）

五、任务 3：紧接普通刷新后的缓存验证

首次页面加载后立即对同一 URL 普通刷新，不强制刷新、不再次清空缓存。浏览器自动加入验证器；帧 28 为第二次 GET，帧 29 为响应，仍属于 `tcp.stream == 1`。

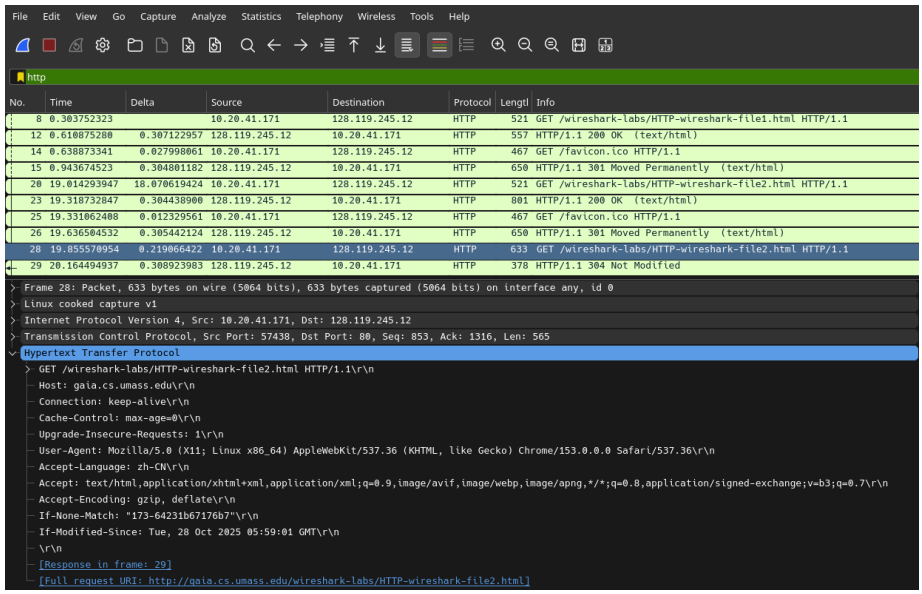


图 7　第二次 GET，帧 28，自动携带条件首部

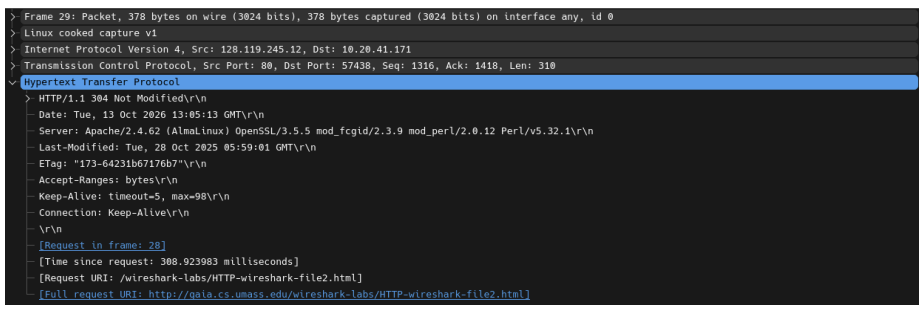


图 8　第二次响应，帧 29，304 Not Modified 且无正文

帧 28 包含 `If-None-Match`、`If-Modified-Since` 和 `Cache-Control: max-age=0`；两项验证器与帧 23 的实际响应值一致。它证明第二次确实发出了网络请求，而非完全依赖本地缓存。

帧 29 没有实体正文，也没有 `Content-Length`、`Content-Type` 或 `File Data` 项。浏览器仍显示首次取得的页面，是复用缓存正文的结果，不能据此认定服务器又发送了完整 HTML。

（接上）实验内容（算法、程序、步骤和方法）	**六、资料查阅与实际排障**
	1. 抓包接口不可用：权限问题
	现象：执行 `tshark -D` 仅列出远程捕获插件，并提示 `Couldn't run dumpcap in child process: Permission denied`。
	排查依据：`/usr/bin/dumpcap` 属于 `wireshark` 组，权限为 `rwxr-xr--`，并具有相应网络捕获能力，但当前会话不属于该组；不是需要以 root 运行整个 Wireshark GUI。
	处理方法：把当前账户加入 `wireshark` 组，在新组会话中运行捕获程序。
	验证结果：重新枚举能看到 `wlo1`、`any`、`lo` 等本机接口，随后成功生成包含 38 帧的 pcapng。
	2. 页面完整显示，不代表第二次又返回 200
	现象：file2 刷新后正文不变，高层页面对象仍呈现成功加载结果。
	排查依据：Wireshark 帧 29 明确显示 `HTTP/1.1 304 Not Modified`，帧 28 验证器与帧 23 一致，且帧 29 不含实体。
	处理与验证：以网络抓包判定服务器实际响应，区分缓存合并后的页面展示。本次结论是“条件 GET 验证成功，304 后复用缓存”，不是“没有请求”或“再次 200 并传输完整正文”。
	自主查阅的资料及其帮助
	（1）ArchWiki：Apache HTTP Server。核对 Linux 包名和配置，理解监听、根目录和目录授权，据此不用 Windows 压缩包，采用独立回环监听实例。
	`https://wiki.archlinux.org/title/Apache_HTTP_Server`
	（2）Apache 2.4 官方 httpd 文档。核对 `-f`、`-t` 和 `-DFOREGROUND`，用于指定配置、语法检查和前台启动。
	`https://httpd.apache.org/docs/2.4/programs/httpd.html`
	（3）Wireshark 官方权限与 dumpcap 文档。帮助定位捕获权限，确认应授权 dumpcap 而非整个 GUI。
	`https://www.wireshark.org/docs/wsdg_html_chunked/ChSrcBinary.html`
	`https://www.wireshark.org/docs/wsug_html_chunked/AppToolsdumpcap.html`
	（4）RFC 9110，第 8.8、13 和 15.4.5 节。用于区分验证器、条件请求和 304 无正文的语义，不把 ETag 误当作必然的内容哈希。
	`https://www.rfc-editor.org/rfc/rfc9110`

数据记录和计算

七、实际报文与关键数据汇总

帧号对应原始`evidence/http-experiment.pcapng`。帧长度包含链路层等开销，不等于 HTTP 实体长度。

帧号	角色	TCP 流	帧长度/B	方法/状态
8	file1 请求	0	521	GET
12	file1 响应	0	557	200 OK
20	file2 首次请求	1	521	GET
23	file2 首次响应	1	801	200 OK
28	file2 第二次请求	1	633	GET
29	file2 第二次响应	1	378	304

客户端	`10.20.41.171`；file1 端口 57426，file2 两次均 57438。
服务器	`128.119.245.12:80`。
HTTP 版本	三组目标交换的请求与响应均为 HTTP/1.1。
语言及可接受编码	`zh-CN`；`gzip, deflate`。
file1 类型、长度	`text/html; charset=UTF-8`；实体 128 字节。
两页最后修改时间	`Tue, 28 Oct 2025 05:59:01 GMT`。
file1 ETag	`"80-64231b6717e87"`。
file2 首次长度	`Content-Length: 371`，导出实体为 371 字节。
file2 ETag	`"173-64231b67176b7"`。
第二次请求验证器	`If-Modified-Since` 与首次 `Last-Modified` 相同；`If-None-Match` 与首次 `ETag` 相同。
第二次响应实体	无实体；未出现 `Content-Length`，不补写缺失首部。

缓存验证的传输影响

第二次 304 没有重新传输 371 字节 HTML，节省的**实体传输量**为

$$371 - 0 = 371 \text{ 字节。}$$

请求及响应首部仍需传输，不能说网络通信完全省去。两份响应帧长度差为 `801-378=423` 字节，包含首部差异，不等于实体节省量。

响应 `Date` 记录 file1 为 `2026-10-13 13:04:53 GMT`，file2 两次分别为 `13:05:12`、`13:05:13 GMT`；终端本地时区与 GMT 不混用。

（接上）数据记录和计算	**八、任务 2 分析题（1—5）** **1. 请求方法、请求资源与 HTTP 版本是什么？** 图 3 帧 8 的完整请求行为： `GET /wireshark-labs/HTTP-wireshark-file1.html HTTP/1.1` 方法为 GET，资源为指定目录的 `HTTP-wireshark-file1.html`，请求版本为 HTTP/1.1。`Host: gaia.cs.umass.edu` 与路径共同确定站点和资源。 **2. 响应版本、状态码与状态短语是什么？** 图 4 帧 12 的状态行为 `HTTP/1.1 200 OK`，所以响应版本为 HTTP/1.1，状态码为 200，状态短语为 OK。它表示本次 GET 成功取得服务器返回的表示，但仍须结合资源路径和正文确认是否为目标页，不能仅凭 200 判断。 **3. 双方 IP 地址和 TCP 端口分别是什么？** 帧 8 的源地址为 `10.20.41.171`、目的地址为 `128.119.245.12`；TCP 源端口 57426，目的端口 80。帧 12 反向使用同一四元组，从 `128.119.245.12:80` 返回 `10.20.41.171:57426`。57426 是该次连接的客户端临时端口，不是固定 HTTP 服务端口。 **4. Accept-Language 包含什么语言，反映什么偏好？** 帧 8 实际只有 `Accept-Language: zh-CN`，表示偏好中国大陆地区使用的简体中文；没有列出英语等其他语言，也没有显式质量权重。该首部影响语言协商，但不保证服务器必然返回中文；本次实验浏览器设置为 zh-CN，目标页实际仍为英文。 **5. Accept-Encoding 与实际采用的内容编码如何区分？** 帧 8 的 `Accept-Encoding: gzip, deflate` 是浏览器愿意接收的内容编码，并非服务器必然采用的编码。判断实际编码应检查**响应**的 `Content-Encoding`。 图 4 帧 12 没有 `Content-Encoding`，正文直接为可读 HTML，表示未施加该首部声明的内容压缩，可按 identity 理解。`charset=UTF-8` 是字符集，不是 gzip 一类的压缩；不能补写抓包中不存在的 br 或 zstd。

（接上）数据记录和计算	**九、任务 2 分析题（6—10）** **6. Accept 与 Content-Type 的含义和区别是什么？** 帧 8 的 Accept 列出 `text/html`、`application/xhtml+xml`、`application/xml;q=0.9`、`image/avif`、`image/webp`、`image/apng`、`*/*;q=0.8` 和 `application/signed-exchange;v=b3;q=0.7`，表达可接受媒体类型和偏好；未写 q 的项默认权重为 1。 帧 12 的 `Content-Type: text/html; charset=UTF-8` 描述**实际返回**的 HTML 及 UTF-8 字符集。Accept 是请求侧可接受的集合，Content-Type 是本次响应表示的类型。 **7. 资源最后修改时间是什么，从哪个首部获得？** 图 4 的 `Last-Modified: Tue, 28 Oct 2025 05:59:01 GMT` 给出资源最后修改时间，即 2025 年 10 月 28 日 05:59:01 GMT。它不是响应生成时间；`Date: Tue, 13 Oct 2026 13:04:53 GMT` 对应本次访问，两者用途不同。 **8. 内容长度是多少，是报文总长还是实体长度？** 帧 12 的 `Content-Length: 128` 指响应实体 128 字节；`File Data: 128 bytes` 与导出的 128 字节 HTML 互相印证。它不含状态行和 HTTP 首部，也不是 TCP/IP 或链路层帧的总长度；该响应的完整捕获帧长度为 557 字节。 **9. 如何确认响应对应目标页而不是其他资源？** 帧 8 的目标路径为 `/wireshark-labs/HTTP-wireshark-file1.html`；帧 12 的 `Request in frame: 8` 和 Full request URI 与之关联。正文含 `Congratulations. You've downloaded the file` 及目标页完整 URL，类型为 HTML、实体 128 字节，因此路径、关联和正文一致。 帧 14 请求 `/favicon.ico`，帧 15 返回 301 并重定向到 HTTPS 图标。这不是目标 HTML 页，应排除，不能用该 301 替代目标页的 200。 **10. 是否允许持久连接，能否仅凭一组交换证明复用？** 双方采用 HTTP/1.1；帧 8 有 `Connection: keep-alive`，帧 12 有 `Connection: Keep-Alive` 和 `Keep-Alive: timeout=5, max=100`，说明允许保持连接。**仅凭帧 8/12 不能证明已复用**，还要观察同一 TCP 流中的后续请求与响应，并确认不是新建连接。 本次帧 14/15 与 8/12 同属流 0，帧 20/23 及 28/29 同属流 1，说明确有连接复用。favicon 的交换可作为复用证据，但不能替代目标页面的内容分析。

（接上）数据记录和计算	**十、任务 3：Conditional GET 分析题（1—4）** **1. 第一次 GET 是否包含缓存验证字段？** 图 5 帧 20 的完整请求没有 `If-Modified-Since` 或 `If-None-Match`。访问前已清除实验浏览器缓存，没有相应缓存副本与验证器可发送，因此发出普通 GET；帧 23 返回 200 和完整 371 字节 HTML。结论依据实际完整首部，而非推测第一次请求必然如此。 **2. 第一次响应有哪些验证字段，分别提供什么信息？** 图 6 帧 23 给出 `Last-Modified: Tue, 28 Oct 2025 05:59:01 GMT` 和 `ETag: "173-64231b67176b7"`。前者记录资源修改时间，供 `If-Modified-Since` 使用；后者是不透明的实体标签，用于标识选中表示的版本，供 `If-None-Match` 比较。 ETag 不必是内容哈希，不能从字符串推断其算法；Last-Modified 也不是浏览器访问时间。浏览器保存响应提供的实际值，作为后续验证条件。 **3. 第二次是否发出 GET，验证器与首次响应有什么关系？** 图 7 帧 28 仍为 `GET /wireshark-labs/HTTP-wireshark-file2.html HTTP/1.1`，证明第二次发出了网络请求。它携带 `Cache-Control: max-age=0`、`If-Modified-Since: Tue, 28 Oct 2025 05:59:01 GMT` 和 `If-None-Match: "173-64231b67176b7"`，两项验证器分别与帧 23 的 Last-Modified、ETag 完全一致。 首部由普通刷新时浏览器自动加入，未手动构造。本次保留已有副本并检查有效性，不是清空缓存或强制刷新后重新下载。 **4. 为什么第二次得到 304，是否验证缓存，怎样影响传输？** 图 8 帧 29 实际返回 `304 Not Modified`，与帧 28 关联，返回的验证器仍与首次响应一致。条件检查表明缓存表示可继续使用，服务器不再次传输实体。**发生了缓存验证，不是没有访问网络**；两项条件同时适用时，以 `If-None-Match` 为优先。 304 没有正文；本次未出现 Content-Length 或 Content-Type，不能补成 `Content-Length: 0`。浏览器使用缓存的 371 字节正文，节省重复实体传输，但仍有第二次请求和 304 首部的开销。资源改变时可能返回 200 及新表示，新鲜缓存也可能不请求网络，但本次证据明确不是这两种情况。 补充：测试页正文出现不规范的 `IN-MODIFIED-SINCE` 拼写，分析以真实首部的标准名称 `If-Modified-Since` 为准；页面加载成功也不能取代抓包中的 304 状态行。

结　论 (结 果)	**1. Web 服务器搭建结果**。 在 Arch Linux 上运行 Apache HTTP Server 2.4.69，完成根目录、默认文档、目录授权与回环地址 8086 端口配置。语法检查、监听状态、浏览器地址栏和 200 日志相互印证，自建首页实际显示了姓名、学号、班级和实验名称。 **2. 基本 HTTP 分析结果**。 file1 的帧 8/12 是一组 GET 与 200 响应，双方采用 HTTP/1.1；路径、首部及正文证明响应为指定页面，实体 128 字节，未声明内容压缩。请求偏好与响应实际类型、编码和长度必须区分。 **3. 缓存验证结果**。 file2 的帧 20/23 先取得 200 和 371 字节实体，普通刷新时帧 28 自动发送验证器，帧 29 返回 304 且无正文，浏览器复用缓存。实际报文验证了条件 GET、缓存验证和连接复用，页面不变不等于没有网络通信。
小　结	**问题 1：依据哪项证据修正或确认了原有判断？** 仅从刷新后仍显示完整页面，容易误判为再次返回 200；帧 28 的条件首部和帧 29 的 304 状态行修正了这一判断。Content-Length、File Data 与整帧长度的对照说明正文大小不能用帧长代替。同样，连接复用应以同一 TCP 流的后续交换证明，不能只看 keep-alive。 **问题 2：若供他人使用，重点核验什么，资料查阅有何帮助？** 应核验监听地址、网络连通性、防火墙、目录权限及最小化暴露，公网使用还需考虑 TLS、日志和安全更新。ArchWiki 与 Apache 文档帮助区别系统默认目录与本次实际配置，Wireshark 文档帮助定位 dumpcap 组权限。关键是把平台版本和本机输出对应起来，而不是照抄 Windows 步骤。
指导老师 评　议	成绩评定：　　指导教师签名：